Symbolic Quarter-Car Dynamics for Active Suspension Design

Derive quarter-car ride and road-holding models from Newton's laws, keep the parameters symbolic, and reuse the resulting transfer functions in active suspension design workflows.

Workflow: free-body equations from first principles, Laplace-domain transfer functions, characteristic polynomial and damping sensitivities, skyhook actuator model, compact Control System Toolbox™ response comparison, `matlabFunction` code generation, generated-function design sweep

Requires: Symbolic Math Toolbox™ and Control System Toolbox

Copyright 2026 The MathWorks, Inc.

1. Quarter-Car Equations of Motion

The 2-DOF quarter-car model captures the vertical motion of one sprung-mass corner and its unsprung wheel/tire assembly. Positive displacement is upward. The suspension deflection is $z_s - z_u$ and the tire deflection is $z_u - z_r$.

$$m_s\ddot{z}_s = -k_s(z_s - z_u) - c_s(\dot{z}_s - \dot{z}_u)$$

$$m_u\ddot{z}_u = k_s(z_s - z_u) + c_s(\dot{z}_s - \dot{z}_u) - k_t(z_u - z_r)$$

```
syms ms ks cs kt positive
mu = sym("mu", "positive");
syms t real
syms zs(t) zu(t) zr(t)
```

Sprung-mass equation:

```
eq1 = ms*diff(zs, t, 2) == -ks*(zs - zu) - cs*(diff(zs, t) - diff(zu, t))
```

```
eq1(t) =
```

$$\mathrm{ms}\,\frac{\partial^2}{\partial t^2}\,\mathrm{zs}(t) = -\mathrm{cs}\left(\frac{\partial}{\partial t}\,\mathrm{zs}(t) - \frac{\partial}{\partial t}\,\mathrm{zu}(t)\right) - \mathrm{ks}\,(\mathrm{zs}(t) - \mathrm{zu}(t))$$

Unsprung-mass equation:

```
eq2 = mu*diff(zu, t, 2) == ks*(zs - zu) + cs*(diff(zs, t) - diff(zu, t)) - kt*(zu - zr)
```

```
eq2(t) =
```

$$\mu \frac{\partial^2}{\partial t^2}\,\mathrm{zu}(t) = \mathrm{cs}\left(\frac{\partial}{\partial t}\,\mathrm{zs}(t) - \frac{\partial}{\partial t}\,\mathrm{zu}(t)\right) + \mathrm{ks}\;(\mathrm{zs}(t) - \mathrm{zu}(t)) + \mathrm{kt}\;(\mathrm{zr}(t) - \mathrm{zu}(t))$$

2. Laplace-Domain Transfer Functions

With zero initial conditions, replace $\dot{z}$ with sZ and $\ddot{z}$ with s^2Z. The symbolic solve gives exact transfer functions from road displacement $Z_r(s)$ to the two performance outputs:

- body acceleration, $H_a(s) = s^2 Z_s(s)/Z_r(s)$
- tire deflection, $H_t(s) = (Z_u(s) - Z_r(s))/Z_r(s)$

```
syms s Zs Zu Zr
eq1_s = ms*s^2*Zs == -ks*(Zs - Zu) - cs*s*(Zs - Zu);
eq2_s = mu*s^2*Zu == ks*(Zs - Zu) + cs*s*(Zs - Zu) - kt*(Zu - Zr);
sol = solve([eq1_s, eq2_s], [Zs, Zu]);
Zs_over_Zr = simplify(sol.Zs / Zr);
Zu_over_Zr = simplify(sol.Zu / Zr);
```

Body acceleration per road displacement:

```
H_body_accel = simplify(s^2 * Zs_over_Zr)
```

```
H_body_accel =
```

$$\frac{\mathrm{kt}\;s^2\;(\mathrm{ks} + \mathrm{cs}\;s)}{\mathrm{ks}\;\mathrm{kt} + \mathrm{cs}\;\mathrm{ms}\;s^3 + \mathrm{cs}\;\mu\;s^3 + \mathrm{ks}\;\mathrm{ms}\;s^2 + \mathrm{kt}\;\mathrm{ms}\;s^2 + \mathrm{ks}\;\mu\;s^2 + \mathrm{ms}\;\mu\;s^4 + \mathrm{cs}\;\mathrm{kt}\;s}$$

Tire deflection per road displacement:

```
H_tire_defl = simplify(Zu_over_Zr - 1)
```

```
H_tire_defl =
```

$$\frac{\mathrm{kt}\;(\mathrm{ms}\;s^2 + \mathrm{cs}\;s + \mathrm{ks})}{\mathrm{ks}\;\mathrm{kt} + \mathrm{cs}\;\mathrm{ms}\;s^3 + \mathrm{cs}\;\mu\;s^3 + \mathrm{ks}\;\mathrm{ms}\;s^2 + \mathrm{kt}\;\mathrm{ms}\;s^2 + \mathrm{ks}\;\mu\;s^2 + \mathrm{ms}\;\mu\;s^4 + \mathrm{cs}\;\mathrm{kt}\;s} - 1$$

3. Characteristic Polynomial

The transfer-function denominator is the fourth-order characteristic polynomial. Keeping the coefficients symbolic shows exactly where mass, tire stiffness, suspension stiffness, and damping enter the dynamics.

```
[~, den_body] = numden(H_body_accel);
```

Characteristic polynomial:

```
den_body = collect(den_body, s)
```

$$\texttt{den_body} = (\mathrm{ms}\ \mu)\ s^4 + (\mathrm{cs}\ \mathrm{ms} + \mathrm{cs}\ \mu)\ s^3 + (\mathrm{ks}\ \mathrm{ms} + \mathrm{kt}\ \mathrm{ms} + \mathrm{ks}\ \mu)\ s^2 + (\mathrm{cs}\ \mathrm{kt})\ s + \mathrm{ks}\ \mathrm{kt}$$

```
coeffs_den = coeffs(den_body, s, 'All');
s3_damping_coeff = simplify(coeffs_den(2))
```

$$\texttt{s3_damping_coeff} = \mathrm{cs}\ (\mathrm{ms} + \mu)$$

4. Reference Vehicle and Frequency Bands

Use representative C-segment sedan quarter-car values. The analysis keeps the symbolic formulas intact, then substitutes these values only for plots and compact downstream checks.

Parameter	Value	Rationale
Sprung mass m_s	320 kg	Quarter-body mass
Unsprung mass m_u	45 kg	Wheel, tire, knuckle, brake
Suspension stiffness k_s	22 kN/m	Passenger-car coil spring
Tire stiffness k_t	200 kN/m	Radial tire vertical stiffness
Actuator time constant τ	0.02 s	Approx. 8 Hz force bandwidth

```
ms_val = 320;
mu_val = 45;
ks_val = 22000;
kt_val = 200000;
```

```
tau_val = 0.02;
omega_s_val = sqrt(ks_val/ms_val);
omega_t_val = sqrt(kt_val/mu_val);
zeta_soft = 0.2;
zeta_stiff = 0.5;
cs_soft = 2*zeta_soft*sqrt(ks_val*ms_val);
cs_stiff = 2*zeta_stiff*sqrt(ks_val*ms_val);
body_band_hz = [0.5, 3.0];
harshness_hz = 10.0;
w_body = logspace(log10(2*pi*body_band_hz(1)), log10(2*pi*body_band_hz(2)), 400);
w_plot = logspace(log10(2*pi*0.5), log10(2*pi*20), 900);
fprintf(['=== Reference Quarter Car ===\n' ...
    'Body-bounce frequency: %.2f Hz\n' ...
    'Wheel-hop frequency:   %.2f Hz\n' ...
    'Soft damping:          zeta = %.2f, c_s = %.0f N*s/m\n' ...
    'Stiff damping:         zeta = %.2f, c_s = %.0f N*s/m\n'], ...
    omega_s_val/(2*pi), ...
    omega_t_val/(2*pi), ...
    zeta_soft, cs_soft, ...
    zeta_stiff, cs_stiff)
```

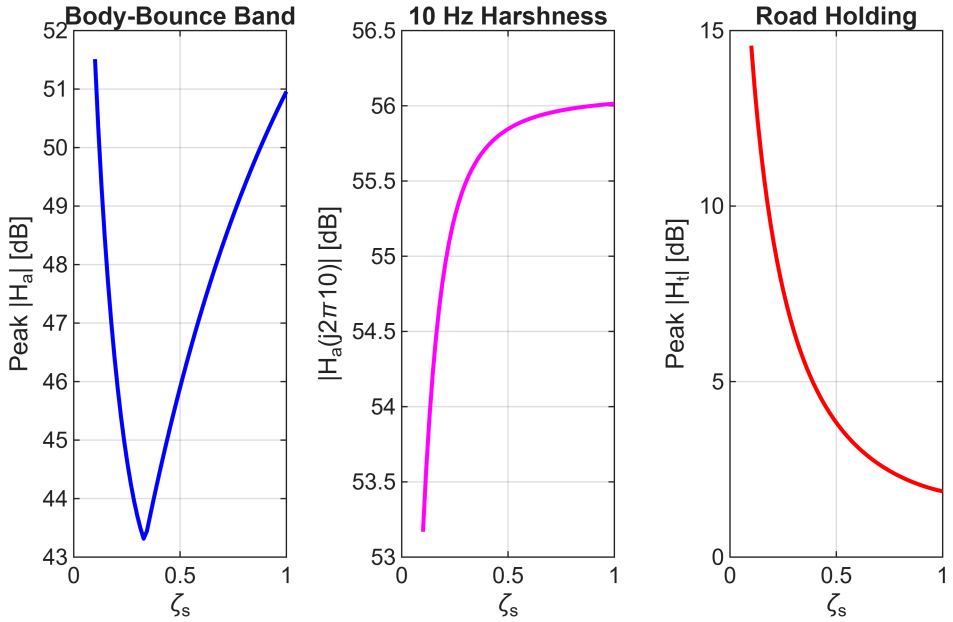
```
=== Reference Quarter Car ===
Body-bounce frequency: 1.32 Hz
Wheel-hop frequency:   10.61 Hz
Soft damping:          zeta = 0.20, c_s = 1061 N*s/m
Stiff damping:         zeta = 0.50, c_s = 2653 N*s/m
```

5. Passive Damping Trade Space

A passive damper is one coefficient trying to satisfy several objectives. Increasing damping reduces body-bounce response and tire deflection, but it also transmits more high-frequency road acceleration into the body. That is the passive compromise exposed by the symbolic derivation.

```
zeta_range = linspace(0.1, 1.0, 60);
syms csky tau positive
symVars = struct("ms", ms, "mu", mu, "ks", ks, "cs", cs, "kt", kt, "s", s, "csky", csky, "tau", tau);
vehicle = struct("ms", ms_val, "mu", mu_val, "ks", ks_val, "kt", kt_val, "tau", tau_val, ...
    "csSoft", cs_soft, "csStiff", cs_stiff, "zetaSoft", zeta_soft, "zetaStiff", zeta_stiff);
bands = struct("wPlot", w_plot, "bodyBandRad", 2*pi*body_band_hz, "bodyBandHz", body_band_hz, ...
```

```
    "harshnessHz", harshness_hz, "peakGainRelTol", 1e-3);
passiveTrade = evaluatePassiveTradeSpace(H_body_accel, H_tire_defl, symVars, vehicle, bands, zeta_range);
plotPassiveTradeSpace(passiveTrade)
```



Symbolic Sensitivity of the Passive Compromise

Replace c_s with $2\zeta_s\sqrt{k_s m_s}$ and differentiate exact magnitude-squared expressions. The signs identify which objectives a larger passive damper helps or hurts at representative frequencies.

```
syms zeta_sym positive
cs_zeta = 2*zeta_sym*sqrt(ks*ms);
H_body_zeta = subs(H_body_accel, cs, cs_zeta);
H_tire_zeta = subs(H_tire_defl, cs, cs_zeta);
H_bounce = subs(H_body_zeta, [ms, mu, ks, kt, s], ...
    [ms_val, mu_val, ks_val, kt_val, 1i*omega_s_val]);
H_harsh = subs(H_body_zeta, [ms, mu, ks, kt, s], ...
    [ms_val, mu_val, ks_val, kt_val, 1i*2*pi*harshness_hz]);
```

```
H_wheelhop = subs(H_tire_zeta, [ms, mu, ks, kt, s], ...
    [ms_val, mu_val, ks_val, kt_val, 1i*omega_t_val]);
[n_bounce, d_bounce] = numden(H_bounce);
[n_harsh, d_harsh] = numden(H_harsh);
[n_wheelhop, d_wheelhop] = numden(H_wheelhop);
mag2_bounce = simplify(expand(n_bounce*conj(n_bounce)) / expand(d_bounce*conj(d_bounce)));
mag2_harsh = simplify(expand(n_harsh*conj(n_harsh)) / expand(d_harsh*conj(d_harsh)));
mag2_wheelhop = simplify(expand(n_wheelhop*conj(n_wheelhop)) / expand(d_wheelhop*conj(d_wheelhop)));
d_bounce_d_zeta = simplify(diff(mag2_bounce, zeta_sym));
d_harsh_d_zeta = simplify(diff(mag2_harsh, zeta_sym));
d_wheelhop_d_zeta = simplify(diff(mag2_wheelhop, zeta_sym));
zeta_eval = 0.3;
sensitivity_names = ["body bounce"; "10 Hz harshness"; "tire wheel-hop"];
sensitivity_values = [
    double(subs(d_bounce_d_zeta, zeta_sym, zeta_eval));
    double(subs(d_harsh_d_zeta, zeta_sym, zeta_eval));
    double(subs(d_wheelhop_d_zeta, zeta_sym, zeta_eval))];
sensitivity_sign = sign(sensitivity_values);
passiveSensitivity = table(sensitivity_names, sensitivity_values, sensitivity_sign, ...
    'VariableNames', {'Metric', 'dMag2_dZeta', 'Sign'})
```

passiveSensitivity = 3×3 table

	Metric	dMag2_dZeta	Sign
1	"body bounce"	-1.0335e+05	-1
2	"10 Hz harshness"	2.8437e+05	1
3	"tire wheel-hop"	-16.3024	-1

6. Skyhook Active Suspension with Actuator Dynamics

A skyhook controller applies an active force that opposes body velocity relative to an inertial reference. The commanded force is $F_{cmd} = -c_{sky}\dot{z}_s$. The actuator is modeled as a first-order lag $G_{act}(s) = 1/(\tau s + 1)$, so the delivered force is bandwidth-limited.

```
syms Fa positive
syms Zs_a Zu_a
eq1_active = ms*s^2*Zs_a == -ks*(Zs_a - Zu_a) - cs*s*(Zs_a - Zu_a) + Fa;
```

```
eq2_active = mu*s^2*Zu_a == ks*(Zs_a - Zu_a) + cs*s*(Zs_a - Zu_a) - kt*(Zu_a - Zr) - Fa;
Fa_actual = -csky*s*Zs_a/(tau*s + 1);
eq1_sky = subs(eq1_active, Fa, Fa_actual);
eq2_sky = subs(eq2_active, Fa, Fa_actual);
sol_sky = solve([eq1_sky, eq2_sky], [Zs_a, Zu_a]);
H_body_sky = simplify(s^2 * sol_sky.Zs_a / Zr);
H_tire_sky = simplify(sol_sky.Zu_a / Zr - 1);
```

Closed-loop body acceleration transfer function:

```
H_body_sky
```

```
H_body_sky =
```

$$\frac{\mathrm{kt}\,s^2\,(\mathrm{ks}+\mathrm{cs}\,s)\,(s\,\tau+1}{\mathrm{ks}\,\mathrm{kt}+\mathrm{cs}\,\mathrm{ms}\,s^3+\mathrm{cs}\,\mu\,s^3+\mathrm{csky}\,\mu\,s^3+\mathrm{ks}\,\mathrm{ms}\,s^2+\mathrm{kt}\,\mathrm{ms}\,s^2+\mathrm{ks}\,\mu\,s^2+\mathrm{ms}\,\mu\,s^4+\mathrm{cs}\,\mathrm{kt}\,s+\mathrm{csky}\,\mathrm{kt}\,s+\mathrm{ks}\,\mathrm{kt}\,s}$$

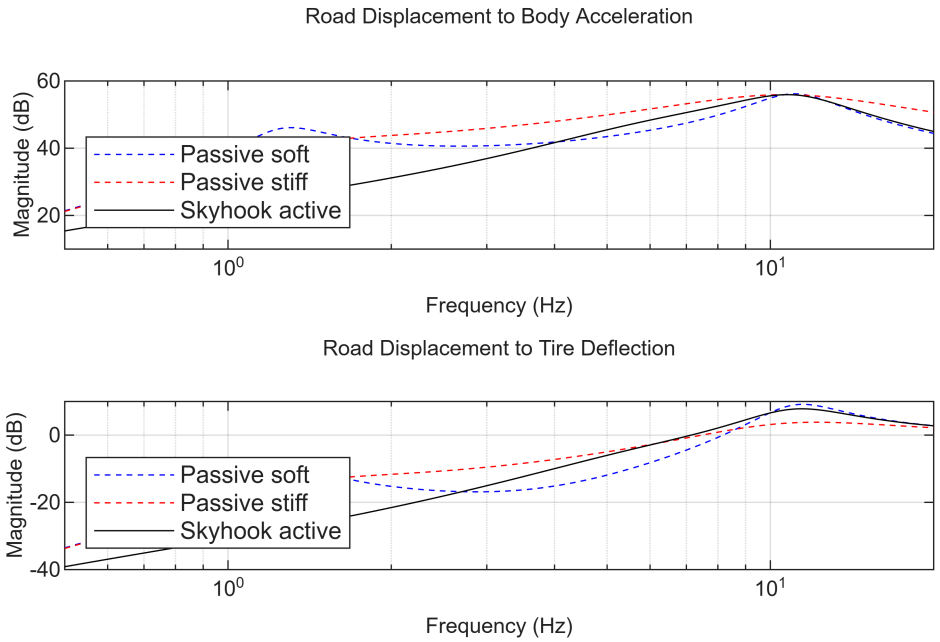
7. Control System Toolbox Reuse

Convert the symbolic transfer functions to named `tf` models, use `getPeakGain` to report continuous-frequency metrics, and use `bodeplot` to draw the final comparison. The downstream analysis is intentionally short; the symbolic derivation remains the source of truth.

```
csky_val = 9000;
comparison = compareSuspensionFrequencyResponses(H_body_accel, H_tire_defl, H_body_sky, H_tire_sky, ...
    symVars, vehicle, bands, csky_val);
bodyChangeDb = comparison.bodyPeakDb(3) - comparison.bodyPeakDb(1);
tireChangeDb = comparison.tirePeakDb(3) - comparison.tirePeakDb(1);
fprintf(['=== Compact Control Reuse ===\n' ...
    'Candidate c_sky:             %.0f N*s/m\n' ...
    'Body-band |H_a| change:      %.1f dB\n' ...
    'Tire |H_t| change:           %.1f dB\n' ...
    'Body-band |H_a| peaks [dB]: soft %.1f, stiff %.1f, skyhook %.1f\n' ...
    'Tire |H_t| peaks [dB]:      soft %.1f, stiff %.1f, skyhook %.1f\n'], ...
    csky_val, ...
    bodyChangeDb, ...
    tireChangeDb, ...
    comparison.bodyPeakDb(1), comparison.bodyPeakDb(2), comparison.bodyPeakDb(3), ...
    comparison.tirePeakDb(1), comparison.tirePeakDb(2), comparison.tirePeakDb(3))
```

```
=== Compact Control Reuse ===
Candidate c_sky:             9000 N*s/m
Body-band |H_a| change:      -9.2 dB
Tire |H_t| change:           -1.3 dB
Body-band |H_a| peaks [dB]: soft 46.1, stiff 45.9, skyhook 36.9
Tire |H_t| peaks [dB]:      soft 9.2, stiff 3.8, skyhook 7.8
```

```
plotSuspensionFrequencyComparison(comparison, bands)
```



8. Code Generation: Reusable Transfer-Function Evaluator

Convert the symbolic body-acceleration transfer function into a deployable MATLAB function. This is the reusable artifact: the derivation is done once, then downstream studies can evaluate the formula directly for arbitrary s.

Free symbols in body acceleration transfer function:

```
bodyAccel_vars = symvar(H_body_accel)
```

`bodyAccel_vars =` [cs ks kt ms μ s]

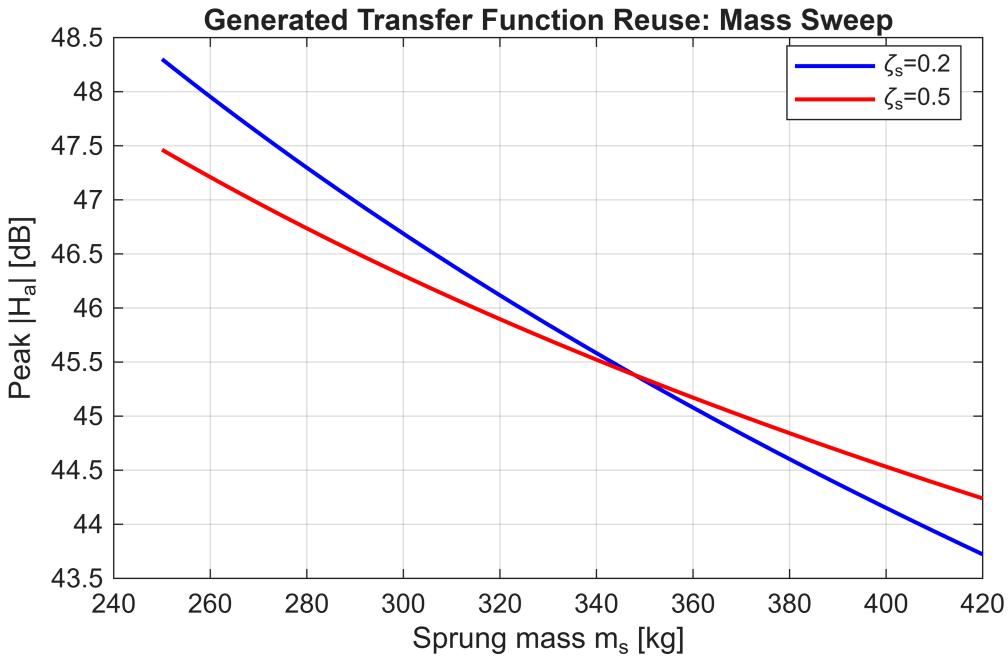
```
matlabFunction(H_body_accel, 'File', 'bodyAccelTF', ...
    'Vars', {s, cs, ks, kt, ms, mu}, 'Outputs', {'H'});
fprintf(['Generated: bodyAccelTF.m\n' ...
    '  Signature: H = bodyAccelTF(s, cs, ks, kt, ms, mu)\n'])
```

```
Generated: bodyAccelTF.m
  Signature: H = bodyAccelTF(s, cs, ks, kt, ms, mu)
```

9. Generated Function Reuse: Platform Mass Sweep

Reuse `bodyAccelTF.m` to evaluate body-bounce-band response over a platform mass range. This is the downstream payoff: the symbolic derivation is done once, then the generated function supports fast variant studies.

```
ms_sweep = linspace(250, 420, 80);
body_peak_soft_mass = zeros(size(ms_sweep));
body_peak_stiff_mass = zeros(size(ms_sweep));
s_grid = 1i*w_body;
for idx = 1:numel(ms_sweep)
    ms_i = ms_sweep(idx);
    cs_soft_i = 2*zeta_soft*sqrt(ks_val*ms_i);
    cs_stiff_i = 2*zeta_stiff*sqrt(ks_val*ms_i);
    body_peak_soft_mass(idx) = max(abs(bodyAccelTF(s_grid, cs_soft_i, ks_val, kt_val, ms_i, mu_val)));
    body_peak_stiff_mass(idx) = max(abs(bodyAccelTF(s_grid, cs_stiff_i, ks_val, kt_val, ms_i, mu_val)));
end
clf
plot(ms_sweep, 20*log10(body_peak_soft_mass), 'b-', ...
    ms_sweep, 20*log10(body_peak_stiff_mass), 'r-', 'LineWidth', 1.5)
xlabel('Sprung mass m_s [kg]')
ylabel('Peak |H_a| [dB]')
title('Generated Transfer Function Reuse: Mass Sweep')
legend('\zeta_s=0.2', '\zeta_s=0.5', 'Location', 'best')
grid on
```



Helper Functions

Local functions for numeric sweeps, Control System Toolbox conversion, and plotting. They are kept out of the main path to keep focus on symbolic derivation and generated-function reuse.

```
function trade = evaluatePassiveTradeSpace(HBody, HTire, symVars, vehicle, bands, zetaRange)
trade.zetaRange = zetaRange;
trade.bodyBandPeak = zeros(size(zetaRange));
trade.bodyHarshness = zeros(size(zetaRange));
trade.tirePeak = zeros(size(zetaRange));
passiveVars = [symVars.ms, symVars.mu, symVars.ks, symVars.cs, symVars.kt];
for idx = 1:numel(zetaRange)
    cs_i = 2*zetaRange(idx)*sqrt(vehicle.ks*vehicle.ms);
    passiveVals = [vehicle.ms, vehicle.mu, vehicle.ks, cs_i, vehicle.kt];
    sysBody = symbolicTfToLti(HBody, passiveVars, passiveVals, "Body acceleration", "m/s^2");
    sysTire = symbolicTfToLti(HTire, passiveVars, passiveVals, "Tire deflection", "m");
    HBody_i = subs(HBody, passiveVars, passiveVals);
```

```
    trade.bodyBandPeak(idx) = getPeakGain(sysBody, bands.peakGainRelTol, bands.bodyBandRad);
    trade.bodyHarshness(idx) = abs(double(subs(HBody_i, symVars.s, 1i*2*pi*bands.harshnessHz)));
    trade.tirePeak(idx) = getPeakGain(sysTire, bands.peakGainRelTol);
end
end
function plotPassiveTradeSpace(trade)
clf
tiledlayout(1,3)
nexttile
plot(trade.zetaRange, 20*log10(trade.bodyBandPeak), 'b-', 'LineWidth', 1.5)
xlabel('\zeta_s')
ylabel('Peak |H_a| [dB]')
title('Body-Bounce Band')
grid on
nexttile
plot(trade.zetaRange, 20*log10(trade.bodyHarshness), 'm-', 'LineWidth', 1.5)
xlabel('\zeta_s')
ylabel('|H_a(j2\pi10)| [dB]')
title('10 Hz Harshness')
grid on
nexttile
plot(trade.zetaRange, 20*log10(trade.tirePeak), 'r-', 'LineWidth', 1.5)
xlabel('\zeta_s')
ylabel('Peak |H_t| [dB]')
title('Road Holding')
grid on
end
function comparison = compareSuspensionFrequencyResponses(HBody, HTire, HBodySky, HTireSky, symVars, vehicle,
bands, cskyVal)
passiveVars = [symVars.ms, symVars.mu, symVars.ks, symVars.cs, symVars.kt];
skyVars = [symVars.ms, symVars.mu, symVars.ks, symVars.cs, symVars.kt, symVars.csky, symVars.tau];
softVals = [vehicle.ms, vehicle.mu, vehicle.ks, vehicle.csSoft, vehicle.kt];
stiffVals = [vehicle.ms, vehicle.mu, vehicle.ks, vehicle.csStiff, vehicle.kt];
skyVals = [vehicle.ms, vehicle.mu, vehicle.ks, vehicle.csSoft, vehicle.kt, cskyVal, vehicle.tau];
comparison.sysBodySoft = symbolicTfToLti(HBody, passiveVars, softVals, "Body acceleration", "m/s^2");
comparison.sysBodyStiff = symbolicTfToLti(HBody, passiveVars, stiffVals, "Body acceleration", "m/s^2");
comparison.sysBodySky = symbolicTfToLti(HBodySky, skyVars, skyVals, "Body acceleration", "m/s^2");
```

```
comparison.sysTireSoft = symbolicTfToLti(HTire, passiveVars, softVals, "Tire deflection", "m");
comparison.sysTireStiff = symbolicTfToLti(HTire, passiveVars, stiffVals, "Tire deflection", "m");
comparison.sysTireSky = symbolicTfToLti(HTireSky, skyVars, skyVals, "Tire deflection", "m");
comparison.sysBodySoft.Name = "Passive soft";
comparison.sysBodyStiff.Name = "Passive stiff";
comparison.sysBodySky.Name = "Skyhook active";
comparison.sysTireSoft.Name = "Passive soft";
comparison.sysTireStiff.Name = "Passive stiff";
comparison.sysTireSky.Name = "Skyhook active";
comparison.bodyPeakDb = 20*log10([
    getPeakGain(comparison.sysBodySoft, bands.peakGainRelTol, bands.bodyBandRad), ...
    getPeakGain(comparison.sysBodyStiff, bands.peakGainRelTol, bands.bodyBandRad), ...
    getPeakGain(comparison.sysBodySky, bands.peakGainRelTol, bands.bodyBandRad)]);
comparison.tirePeakDb = 20*log10([
    getPeakGain(comparison.sysTireSoft, bands.peakGainRelTol), ...
    getPeakGain(comparison.sysTireStiff, bands.peakGainRelTol), ...
    getPeakGain(comparison.sysTireSky, bands.peakGainRelTol)]);
end
function plotSuspensionFrequencyComparison(comparison, bands)
clf
plotOptions = bodeoptions;
plotOptions.PhaseVisible = 'off';
plotOptions.FreqUnits = 'Hz';
plotOptions.MagUnits = 'dB';
plotOptions.Grid = 'on';
bodySoftPlot = comparison.sysBodySoft;
bodyStiffPlot = comparison.sysBodyStiff;
bodySkyPlot = comparison.sysBodySky;
tireSoftPlot = comparison.sysTireSoft;
tireStiffPlot = comparison.sysTireStiff;
tireSkyPlot = comparison.sysTireSky;
bodySoftPlot.InputName = "";
bodyStiffPlot.InputName = "";
bodySkyPlot.InputName = "";
tireSoftPlot.InputName = "";
tireStiffPlot.InputName = "";
tireSkyPlot.InputName = "";
```

```
bodySoftPlot.OutputName = "";
bodyStiffPlot.OutputName = "";
bodySkyPlot.OutputName = "";
tireSoftPlot.OutputName = "";
tireStiffPlot.OutputName = "";
tireSkyPlot.OutputName = "";
tiledlayout(2,1)
axBody = nexttile;
bodyPlot = bodeplot(axBody, ...
    bodySoftPlot, 'b--', ...
    bodyStiffPlot, 'r--', ...
    bodySkyPlot, 'k-', ...
    bands.wPlot, plotOptions);
bodyPlot.Title.String = "Road Displacement to Body Acceleration";
bodyPlot.Title.Visible = 'on';
bodyPlot.Legend.Visible = 'on';
bodyPlot.Legend.Location = 'southwest';
axTire = nexttile;
tirePlot = bodeplot(axTire, ...
    tireSoftPlot, 'b--', ...
    tireStiffPlot, 'r--', ...
    tireSkyPlot, 'k-', ...
    bands.wPlot, plotOptions);
tirePlot.Title.String = "Road Displacement to Tire Deflection";
tirePlot.Title.Visible = 'on';
tirePlot.Legend.Visible = 'on';
tirePlot.Legend.Location = 'southwest';
end
function sys = symbolicTfToLti(expr, vars, vals, outputName, outputUnit)
exprVal = subs(expr, vars, vals);
[numExpr, denExpr] = numden(exprVal);
sys = tf(sym2poly(numExpr), sym2poly(denExpr), ...
    InputName="Road displacement", InputUnit="m", OutputName=outputName, OutputUnit=outputUnit);
end
```